

Флаговете на състояние, наречени още аритметични флагове, отразяват особеностите на изпълнение на инструкциите. Назначението на тези флагове е:

- CF (Carry Flag) - флаг за пренос. Вдига се в единица при пренос или заем от старшия бит на резултата;

- PF (Parity Flag) - флаг за четност. Установява се в 1, ако младшият байт на резултата съдържа четен брой единични битовете;

- AF (Auxiliary carry Flag) - флаг за спомагателен пренос. Установява се в единица при пренос или заем от младшата тетрада на резултата;

- ZF (Zero Flag) - флаг за нула. Установява се в единица при нулев резултат;

- SF (Sign Flag) - флаг за знак. Състоянието му съвпада със състоянието на старшия (знаков) бит на резултата;

- OF (Overflow Flag) - флаг за препълване. Установява се в единица, ако резултатът излиза извън диапазона за представимост на числата (прекалено голямо положително или твърде малко отрицателно число).

От флаговете за състояние само бит CF може да се модифицира директно с инструкциите *stc*, *clc* и *cmc*. Един или повече флагове на състояние се използват от инструкциите: за условен преход *Jху*, условно установяване на байт *SETху*, условни цикли *LOOPху* и условен обмен на данни *CMOVху*, където с *х* е обозначен местваният флаг(ове), а с *у* - проверяваното състояние на този флаг(ове).

- DF (Direction Flag) - флаг за посока. Този флаг задава посоката при обработка на низове с инструкциите *mows*, *cmps*, *scas*, *lods*, *stos*, *ins*, *outs*. След изпълнение на инструкция за обработка на низове при DF=0 автоматично се инкрементират, а при DF=1 автоматично се декрементират индексните регистри ESI и EDI, адресиращи низовете. Флагът DF се установява в единица с инструкцията *std* и нулира с инструкцията *cld*.

Системните флагове въздействат върху различни аспекти от работата на процесора. Те управляват входа и изхода, маскирането на апаратните прекъсвания, тестването, превключването на задачите и виртуалния режим:

- TF (Trace Flag) - флаг за тестване. При TF=1 се задава стъпков режим на работа на процесора, т.е. след изпълнение на всяка инструкция се генерира изключение за тестване;

- IF (Interrupt Flag) - флаг за прекъсване. При нулиране на IF се забранява обработката на маскируемите прекъсвания по вход INTR. Върху флага IF може да се въздейства директно с инструкциите *cli* и *sti*, и индиректно с инструкциите *popf*, *popfd* и *iret*;

- IOPL (I/O Privilege Level) - двубитово поле, задаващо нивото на привилегия за достъп до входно-изходното пространство. Изпълняващата се задача може да изпълнява операции за вход-изход само, ако има ниво на привилегия, по-високо или равно от нивото, зададено чрез IOPL. Това поле е съставна част от механизма на защита на МП и действа в Р-режим. То може да бъде модифицирано само неявно с инструкциите *popf* и *iret*;

- NT (Nested Task) - флаг за вложена задача. Флагът NT действа в Р-режим и управлява верижното извикване и прекъсване на задачите. МП вдига този флаг при извикване на задача с инструкцията *call*, чрез прекъсване или чрез изключение. Състоянието на NT се проверява и модифицира при връщане от задача с инструкцията *iret*. При NT=1 се осъществява превключване на задача, а при NT=0 се изпълнява нормално връщане от прекъсване;

- RF (Resume Flag) - флаг за възобновяване. Управлява реакцията на МП по отношение на обработката на изключителни събития при тестване. Флагът RF действа съвместно с регистрите за тестване;

- VM (Virtual Mode) - виртуален режим. При VM=0 процесорът може да работи или в R- или в Р-режим, а при VM=1 преминава в V-режим и емулира програмната среда на МП I8086;

- AC (Alignment Check) - контрол на изравняването. Установяването в 1 на флаг AC (при вдигнат флаг AM на управляващия регистър CR0) разрешава контрола за изравняване при обръщение към паметта. Изключение възниква при обръщение към неизравнен операнд (например 16-разреден операнд, разположен на нечетен адрес или 32-разреден операнд, неразположен на адрес кратен на 4);

- VIF (Virtual Interrupt) - флаг за виртуално прекъсване. Съдържа виртуалния образ на флага IF. Използва се в съчетание с флага VIP. МП разпознава флага VIF само ако или флагът VME или флагът PVI в управляващия регистър CR4 са установени в 1 и значението на IOPL е по-малко от 3;

- VIP (Virtual Interrupt Pending) - флаг за прието виртуално прекъсване. Установява се програмно и индицира, че има прието, но още необработено виртуално прекъсване. МП чете, но не модифицира флага VIP. Използва се в съчетание с флага VIF и за него важат същите условия за разпознаване.

Нулирането на този флаг означава, че няма необработени прекъсвания, генерирани от задачите в разширения виртуален режим VME (Virtual Mode Extension);

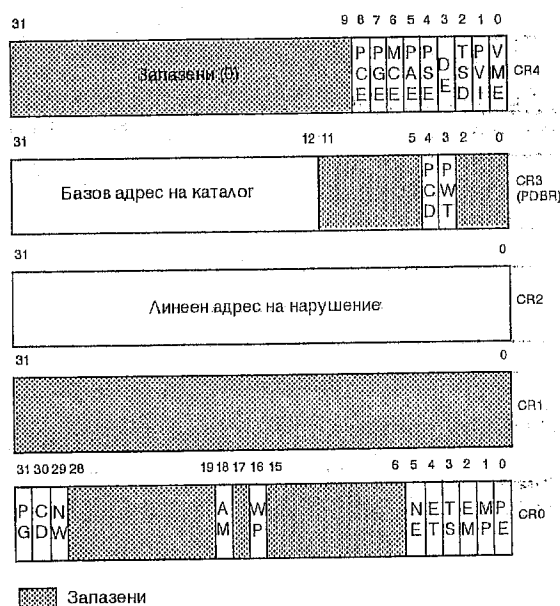
- ID (Identification) - флаг за идентификация. Възможността за програмна промяна на този флаг означава, че МП поддържа инструкция *cpuid* за идентификация на процесора.

Флаговете VIP и VIF са въведени за първи път в Pentium и са достъпни само за процесори, които поддържат разширен виртуален режим VME.

2.2. Управляващи регистри

Управляващите регистри (фиг.5.2) служат за определяне режима на работа на МП и състоянието на вътрешните му компоненти (FPU, кеш, устройство за странично преобразуване). Те включват:

- CR0 - съдържа системни управляващи флагове, които управляват режима на работа и състоянието на процесора;
- CR1 - резервиран за бъдещи разширения;
- CR2 - съдържа линейния адрес, предизвикал последното странично нарушение;
- CR3 - съдържа физическия базов адрес на каталога на страниците. Флаговете PCD и PWT управляват кеширането на каталога във вътрешната кеш памет за данни;
- CR4 - съдържа група флагове, разрешаващи някои архитектурни разширения в Р-режим.



Фиг. 5.2. Управляващи регистри

В защитен режим управляващите регистри могат да бъдат четени с инструкцията *move* от всяко привилегировано ниво. Зареждането им с ново съдържание обаче може да става само от най-високото (нулево) ниво на привилегия. Флаговете в регистрите CR0 и CR4 имат следното функционално предназначение:

- флагове за управление на страницирането:
 - PG (Paging) - CR0[31]. Разрешава (PG=1) или забранява (PG=0) механизма на странициране в защитен режим. Вдигането на PG в друг режим (PE=0) предизвиква изключение #GP (нарушение на защитата);
 - PSE (Page Size Extensions) - CR4[4]. Установен в 1 флагът разрешава използване на страници с размер 4MB;
 - PGE (Page Global Enable) - CR4[7]. Разрешава (PGE=1) използването на глобални страници. Въведен е в процесорите P6;

- флагове за управление режимите на кеширане:
 - CD (Cache Disable) - CR0[30]. При нулирани флагове CD и NW кеширането във вътрешните кеш памети на МП е разрешено;
 - NW (Not Write-through) - CR0[29]. При нулирани флагове CD и NW при попадение в кеш за Pentium и P6 е разрешен обратен запис, а за I486 - едновременен запис в паметта;
- флагове за работа с паметта:
 - AM (Alignment Mask) - CR0[18]. При AM=1 е разрешен контрол за изравняване на операндите в паметта. Необходимото допълнително условие за тази цел е: AC=1, текущо ниво на привилегия - 3 и P- или V-режим на работа на МП;
 - WP (Write Protect) - CR0[16]. Забранява (WP=1) на системни програми (с привилегия супервайзор) да извършват запис в разрешени само за четене потребителски страници;
 - PAE (Physical Address Extension) - CR4[5]. Разрешава (PAE=1) 36 бита физически адрес;
- флагове за разширение на виртуалния режим:
 - VME (Virtual-8086 Mode Extensions) - CR4[0]. Разрешава (VME=1) подобрена обработка на прекъсвания и изключения за V-режим с цел повишаване на производителността. Осигурява апаратно поддържане на флага VIF за увеличаване на надеждността при изпълнение на програми за I8086 в мултизадачна и мултипроцесорна среда;
 - PVI (Protected Mode Virtual Interrupts) - CR4[1]. Разрешава (PVI=1) в защитен режим действието на флага VIF.
- флагове за работа с копроцесор:
 - NE (Numeric Error) - CR0[5]. Разрешава (NE=1) вътрешния механизъм за съобщаване за грешка от вътрешното устройство с плаваща запетая FPU. При NE=0 се установява PC стил за съобщаване на грешка от копроцесор чрез сигналите на МП IGNNE# и FERR#;
 - ET (Extension Type) - CR0[4]. Не се използва в Pentium и P6, а в I486 и I386 индицира поддържане на копроцесор I387DX;
 - EM (Emulation) - CR0[2]. EM=1 индицира отсъствие, както на вътрешно FPU, така и на външен копроцесор. Инструкциите с плаваща запетая и MMX не се разпознават и предизвикват изключение #UD (невалиден код на операция);
 - MP (Monitor Coprocessor) - CR0[1]. Управлява генерирането на изключение #NM (недостъпно устройство) при инструкциите *wait* и *fwait* в зависимост от състоянието на флага TS;
- други флагове:
 - PE (Protection Enable) - CR0[0]. При PE=1 разрешава защитен режим, а при PE=0 - реален режим;
 - TS (Task Switched) - CR0[3]. МП вдига флага TS при всяко превключване на задача. Този флаг позволява да се задържи съхраняването на контекста на FPU до появата на инструкция с плаваща запетая в новата задача;
 - TSD (Time Stamp Disable) - CR4[2]. TSD=1 разрешава използването на инструкцията *rdtsc* само от нулево ниво на привилегия, а при TSD=0 тази инструкция може да бъде използвана от всяко привилегировано ниво. Инструкцията *rdtsc* чете в регистрите EDX:EAX брояча на време;
 - * MCE (Machine Check Enable) - CR4[6]. Разрешава (MCE=1) генериране на изключение от схемите за машинен контрол;
 - DE (Debugging Extension) - CR4[3]. Разширява възможностите на МП за тестване.

2.3. Специфични за модела на микропроцесора регистри

В архитектурата на Intel е включен масив от 64 разредни регистри със специализирано предназначение, наречени MSR (Model Specific Registers). MSR са достъпни главно за ОС, т.е. за програми, изпълнявани на привилегировано ниво 0. Тези регистри се използват за разширени възможности за тестване, за наблюдение на производителността на процесора, за целите на вътрешния контрол на процесора, за разширени възможности за управление на паметта чрез обособяване на различни типове области в нея. Броят и функциите на регистрите MSR варират в различните членове на фамилията Intel.

3. Управление на паметта в защитен режим

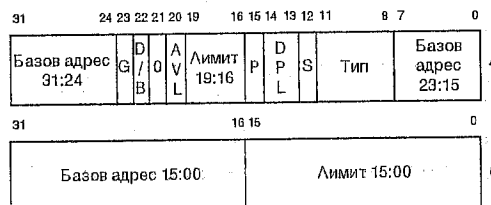
В Intel архитектурата се използват два механизма за управление на паметта - сегментиране и странициране. И двата механизма целят да осигурят изолиране на адресните пространства на отделните задачи (програми) в многозадачна (мултипрограмна) среда. Използваните защити се базират на нива на привилегии, тип на съхраняваната информация, права на достъп, размер на адресираната област.

3.1. Сегментация на паметта

Адресируемата област на МП, наречена линейно адресно пространство, се разделя на по-малки защитени области - сегменти. В мултипрограмна среда всяка програма работи със собствен набор сегменти. Всеки сегмент се характеризира с набор атрибути, които задават разположението и заемания обем в паметта, тип и права на достъп до информацията в него, ниво на привилегия и др. С всеки сегмент е свързан дескриптор, съхраняващ значението на неговите атрибути.

3.1.1. Сегментни дескриптори

Достъпът до данните в даден сегмент се осъществява чрез неговия дескриптор. Дескрипторът е 8 байтова структура, която съдържа следната информация за сегмента (фиг.5.3):



Фиг. 5.3. Структури от данни за сегментно преобразуване на адреса.

- базов адрес (байтове 2,3,4 и 7). Задава началния адрес на сегмента в 4GB линейно адресно пространство на МП. С цел оптимизация на производителността се препоръчва този адрес да бъде изравнен на 16 разредна граница;

- лимит. 20-разредният лимит задава размера на сегмента, който се изчислява в зависимост от значението на бита за грануларност G (Granularity). При G=0 размерът на сегмента е в границите от 1 байт до 1 MB. При G=1 размерът се изчислява със стъпка 1 страница - 4KB и е в границите от 4KB до 4GB, т.е. може да покрие цялото линейно адресно пространство. МП използва лимита в зависимост от типа на сегмента - даннов или стеков. При даннов сегмент допустимият адрес е в границите $\text{базов адрес} + \text{базов адрес} + \text{лимит}$. При стеков сегмент допустимият адрес е в границите $\text{максимален адрес} - \text{базов адрес} + \text{лимит}$.

- тип. Определя типа на достъп до информацията в сегмента и посоката на нарастването му. Интерпретацията на това поле зависи от типа на дескриптора, задаван чрез бита S (System): S=0 - системен дескриптор, S=1 - дескриптор на даннов или кодов сегмент. Полето за тип при S=1 задава следните права за достъп:

- 000B - сегмент данни, разрешен само за четене
- 001B - сегмент данни, разрешен за четене и запис
- 010B - стеков сегмент, разрешен само за четене (на практика не се използва)
- 011B - стеков сегмент, разрешен за четене и запис
- 100B - кодов сегмент, разрешен само за изпълнение
- 101B - кодов сегмент, разрешен за изпълнение и четене
- 110B - подчинен кодов сегмент, разрешен само за изпълнение
- 111B - подчинен кодов сегмент, разрешен за изпълнение и четене

В регистър CS например, не трябва да се зареждат селектори към сегменти с типове 0+3, тъй като това са даннови сегменти. Някоя програма не може да модифицира сегмент, разрешен само за четене. Сегментите с код, за които е разрешено само изпълнение, не могат да се четат, което преняства несанкционираното копиране на програми. Нарушаването на тези правила предизвиква изключение #GP (нарушение на защитата).

Стековите сегменти се различават от сегментите с данни с посоката на разширение на адреса

и интерпретацията на лимита. При стековите сегменти адресът се разширява надолу - от големи към малки адреси, но трябва да бъде по-голям от лимита на сегмента.

- достъп - бит A (Accessed). Битът A се установява в 1 автоматично от МП, когато се осъществява обръщение към сегмента. Използва се от ОС за избиране на сегментите за извеждане върху диска при сполдинг;

- ниво на привилегия DPL (Descriptor Privilege Level). Задава нивото на привилегия на сегмента, което може да бъде в диапазона от 0 (най-висока привилегия) до 3 (най-ниска привилегия);

- присъствие P (Present). Индицира дали сегментът е резидентен (присъстващ) в паметта (P=1) или не (P=0). При зареждане в селекторен регистър на селектор към нерезидентен сегмент се генерира изключение #NP (отсъстващ сегмент). В полетата за базов адрес и лимит в дескриптора на нерезидентен сегмент ОС може да записва системна информация за сегмента;

- размер по премълчаване D (Default size). Въведен е за съвместимост с I286 и се интерпретира в зависимост от типа на сегмента. За даннов сегмент D=1 означава 32 разредни операнди, а D=0 - 16 разредни операнди. При кодов сегмент D определя размера по премълчаване на ефективния адрес и размера на адресираните операнди. D=1 определя 32 разреден адрес и 32/8 разредни операнди, а D=0 - 16 разреден адрес и 16/8 разредни операнди. За промяна на значението по премълчаване се използват префиксите 66h и 67h. За стековите сегменти D определя размера на стековия указател, използван при неявните операции със стек - *push*, *pop*, *call*. D=1 определя 32 разреден указател, съхраняван в ESP и горна граница на стека 4GB (FFFFFFFFh), а D=0 - 16 разреден указател в SP и горна граница 64KB (FFFFh);

- свободен за използване от системата бит AVL (AVAILable).

Сегментните дескриптори типично се създават от компилиращи, свързващи и зареждащи програми или от ОС, но не и от приложните програми.

3.1.2. Дескрипторни таблици

Дескрипторите на сегментите за удобство при ползване се съхраняват в специални структури, наречени дескрипторни таблици. Максималният размер на една таблица е 64KB, т.е. 8192 различни дескриптора. В МП се използват три типа дескрипторни таблици:

GDT (Global Descriptor Table) - глобална дескрипторна таблица. Тази общосистемна таблица съдържа дескрипторите на общите сегменти, които могат да се използват от всички задачи в системата. За локализация на таблицата GDT се използва специален регистър GDTR, който съдържа 32 разредния линеен базов адрес на таблицата и 16 разреден лимит. Първият елемент на GDT, имащ нулев сегментен селектор, не се използва от МП. Зареждането на сегментните регистри CS и SS с нулев селектор предизвиква изключение #GP. Опитът да се използва нулев селектор, зареден в другите селекторни регистри, също предизвиква изключение.

IDT (Interrupt Descriptor Table) - дескрипторна таблица на прекъсванията. Общосистемна таблица, съдържаща дескрипторите на специални системни обекти (S=0 в дескрипторите). Тези обекти, наречени шлюзове, определят входните точки в процедурите за обработка на прекъсвания и изключения. За локализация на таблицата IDT се използва специален регистър IDTR, който съдържа 32 разредния линеен базов адрес на таблицата и 16 разреден лимит.

LDT (Local Descriptor Table) - локална дескрипторна таблица. В мултизадачна система за всяка задача може да се построи локална дескрипторна таблица. Тя определя сегментите, достъпни само за тази конкретна задача. За съхранение на таблиците LDT се използват сегменти, чиито дескриптори се съхраняват в таблицата GDT. За локализация на таблицата LDT е предназначен 16 разреден регистър LDTR, който съдържа селектора на дескриптора на LDT на текущата активна задача. С LDTR е свързан разширителен регистър, наречен кеш-регистър, в който се съхранява дескриптора на LDT на текущата задача.

С регистрите на дескрипторните таблици оперират следните привилегировани инструкции на МП: *lgdt m48* и *lidt m48* зареждат регистрите GDTR и IDTR съответно, а инструкциите *sgdt m48* и *sidt m48* съхраняват съдържанието на тези регистри в паметта. Операндите на тези инструкции са 48 разредни данни в паметта, младшите 16 разряда на които задават лимита, а останалите 32 разряда - линеен базов адрес на съответната таблица. Инструкциите *lldt r16/m16* и *sldt r16/mem16* служат съответно за зареждане и съхраняване на *ldtr*. Операндът е 16 разреден регистър или 16 разредни данни в паметта, задаващи валиден или празен (0000h) селектор.

3.1.3. Сегментни селектори

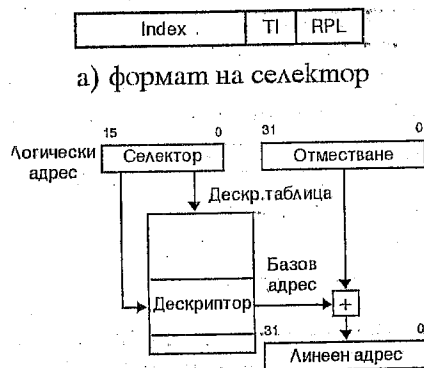
Достъпът до дескрипторите се осъществява чрез 16 разредните сегментни регистри на МП. Съдържанието на сегментен регистър се нарича селектор (фиг. 5.4 а). Двама младши разреда на селектора определят заявеното ниво на привилегия (RPL), което се използва в механизма на защита. Битът TI (Table Indicator) определя таблицата, в която е разположен дескрипторът: TI=0 указва таблицата GDT, а TI=1 - таблицата LDT. Старшите 13 разреда на селектора (поле Index) определят нужния дескриптор в дескрипторната таблица.

3.1.4. Сегментно преобразуване на адреса

Преобразуването на адреса преминава през следната последователност от действия:

- изчисляване на ефективния (логически) адрес;
- определяне на дескрипторната таблица и номера на дескриптора от селектора в съответния сегментен регистър;
- извличане на дескриптора на сегмент от адрес базов_адрес(GDT/LDT)+Index*2³;
- получаване на линейния адрес чрез сумиране на ефективния адрес и базовия адрес на сегмента от неговия дескриптор.

За да се избегнат обръщенията към паметта за извличане на дескриптора на сегмента при следващите обръщения към него, се използва кеширане на дескрипторите. За целта с всеки сегментен регистър, както и с регистъра LDTR, е асоцииран 64 разреден кеш-регистър, наречен още регистър-сянка. Кеш-регистрите са невидими и недостъпни за програмите. При зареждане на селектор в сегментен регистър МП автоматично извлича и зарежда в неговия кеш-регистър съответния дескриптор.



а) формат на селектор
б) формиране на линейен адрес
Фиг. 4.4. Сегментно преобразуване

3.1.5. Инструкции за работа с дескриптори

Значенията на селекторите в сегментните регистри са достъпни на програмата, а чрез тях стават достъпни и дескрипторите на съответните сегменти. За опростяване на анализа на атрибутите на достъпните за дадена програма сегменти са предвидени специални инструкции:

lar r16/32, r16/m16 - зарежда в регистъра-получател правата на достъп на сегмента (разреди 47-40 и битовете G, D, X U на дескриптора), указван от адресирания с операнда-източник селектор.

lsl r16/32, r16/m16 - зарежда в регистъра-получател лимита на сегмента, указван от адресирания с операнда-източник селектор.

verr r16/32 - проверява достъпността за четене на сегмента, указан от адресирания селектор.

verw r16/32 - проверява достъпността за запис на сегмента, указан от адресирания селектор.

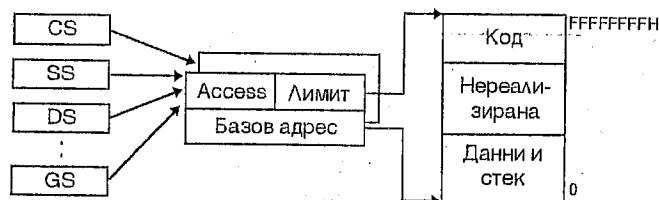
Инструкциите **verr** и **verw** вдигат флага ZF=1, ако е разрешен съответното право на достъп.

3.1.6. Сегментни модели

Основните сегментни модели за организация на паметта са:

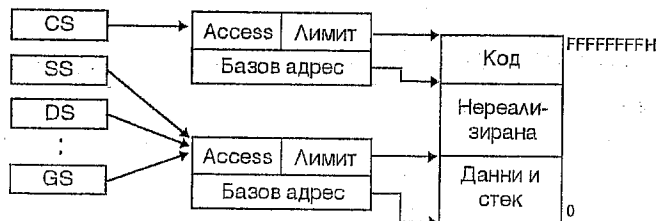
Базов плосък модел. Това е най-прост за реализация модел, при който системните и приложни програми имат достъп до непрекъснато, несегментирано адресно пространство. Моделът скрива

механизма на сегментиране, както за приложните, така и за системните програмисти. За реализацията му са необходими поне два сегментни дескриптора - за достъп до кодов сегмент и за достъп до даннов сегмент. И двата сегмента обхващат цялото линейно пространство, т.е. имат една и съща стойност за базов адрес - 0 и лимит - 4GB (фиг. 5.5). Не се осъществява апаратна защита при непопадане на адрес в реална физически достъпна област памет.



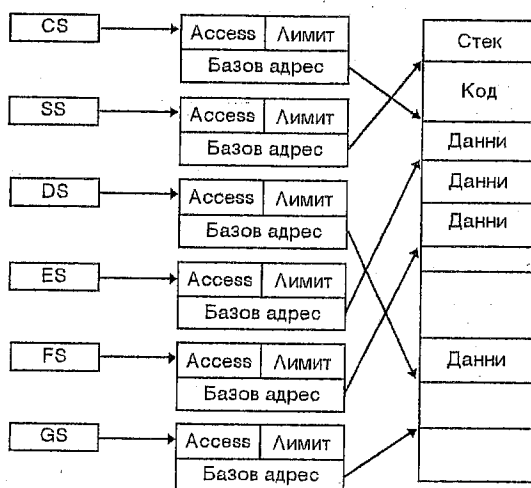
Фиг. 5.5. Базов плосък модел

Защитен плосък модел. Подобен е на базовия, но в сегментите се включват само области от адреси, реално съществуващи във физическата памет (фиг. 5.6). При попадение в несъществуваща област се генерира изключение #GP (общо нарушение на защитата). Моделът може да бъде усложнен за осигуряване на по-добри възможности за защита. Например чрез механизма на странициране може да се осигури изолация на системното от потребителското адресно пространство. Чрез добавяне на различни структури от страници за различните задачи се постига защита на приложенията едно от друго. Такъв подход се използва в редица популярни операционни системи (например Unix).



Фиг. 5.6. Защитен плосък модел

Многосегментен модел. При него се използват пълните възможности на механизма на сегментиране за осигуряване на апаратна защита на код, даннови структури и задачи. Всяка програма (задача) има на разположение собствена таблица от сегментни дескриптори (LDT) и свои собствени сегменти. Достъпът до всички сегменти се контролира апаратно.



Фиг. 5.7. Многосегментен модел

3.2. Странична организация на паметта

В защитен режим процесорите поддържат странично преобразуване на адреса, което внася второ ниво на преобразуване на логическия адрес във физически. Основното приложение на страничното преобразуване е свързано с организацията на виртуална памет, която позволява достъп до по-голямо пространство памет от реално реализираната физическа памет посредством използване на дисково пространство.

За разлика от сегментирането, при което базовият обект памет е блок с произволен размер, наречен сегмент, страницирането използва като базов обект памет блок с фиксиран размер, наречен страница. Ако се използва само сегментиране, структурите от данни, представени във физическата памет трябва да съдържат всички си елементи в паметта. Ако се използва и странициране, структурите могат да бъдат частично в паметта и частично върху диска. Страницирането е напълно невидим за приложните програми механизъм. За страниците също се прилагат защитни механизми: две нива на привилегия и защита от запис.

При разрешено странично преобразуване линейното адресно пространство на МП (4GB виртуална памет) се разделя на 1M страници по 4KB. Физическата памет също се разделя на страници със същия размер, наречени странични кадри. Виртуалната памет е много по-голяма от физически реализираната памет. Следователно във всеки един момент само част от всички страници на виртуалната памет присъстват във физическата памет. Всяка виртуална страница може да бъде заредена във всяка физическа страница. Процесорът автоматично генерира изключение #PF (странично нарушение) при обръщение към страница неприсъстваща във физическата памет. При обработката на това изключение ОС зарежда нужната страница от външната памет. Страницата става достъпна посредством базовия адрес на зареждането ѝ в паметта.

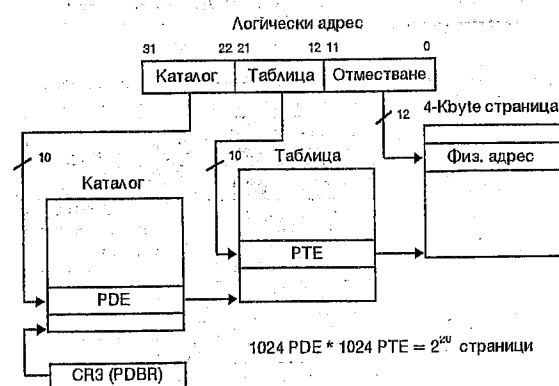
3.2.1. Информационни структури

Информацията, която МП използва, за да преобразува линейните адреси във физически е включена в данни структури на две йерархични нива (фиг. 5.8):

- каталог на страниците, представляващ масив (таблица) от 1024 32-разредни елемента - PDE (Page Directory Entry). Всеки PDE адресира подчинена таблица на страници. Каталогът на страниците е достъпен чрез управляващия регистър CR3, наречен PDBR (Page Directory Base Register). PDBR съдържа 20-разредния базов физически адрес на каталога.

- таблица на страници. Масив от 1024 32-разредни елемента - PTE (Page Table Entry), всеки от които адресира физическа страница в паметта.

Каталогът и таблиците на страниците са с размер една 4KB страница.



Фиг. 5.8. Двуетапно преобразуване на линейния адрес във физически

Елементите PDE и PTE имат еднакъв формат (фиг. 4.9) и задават следните атрибути на страниците:

- присъствие в оперативната памет - бит P (Present). При P=1 страницата е в паметта и е валиден форматът на PTE/PDE от фиг.5.9, а при P=0 страницата не е в паметта и останалата част от PTE/PDE е достъпна за ОС за съхранение на системна информация. В случай на обръщение към PTE/PDE, съдържащ P=0, се генерира изключение #PF;

В съвременната Intel архитектура страницирането се управлява от три флага:

- PG (Paging) от управляващия регистър CR0 (CR0[31]) разрешава механизма на странициране.

Въведен е от I386;

- PSE (Page Size Extension) в управляващия регистър CR4 (CR4[4]) разрешава използване на страници с размер 4MB и 2MB. Въведен е от Pentium;

- PAE (Physical Address Extension) в управляващия регистър CR4 (CR4[5]) разрешава разширение физическия адрес от 32 на 36 разряда. Въведен е от Pentium Pro.

В режим на 32-разряден адрес са допустими страници с размер 4KB и 4MB, а в режим на разширена адресация - 4KB и 2MB. Кой от двата размера е използван за дадена страница се указва от бита PS (Page size) на PDE: PS=0 - задава таблица от 4KB-ви страници, а PS=1 - задава 4MB-ва (при 32-разряден адрес) или 2MB-ва (при 36 разряден адрес) страница.

При разширена адресация преобразуването на виртуалния адрес се изпълнява на три нива, като се въвежда допълнителна структура, наречена таблица от указатели към каталози на страници (Page_Directory_Pointer_Table - PDPT). Каталогите и таблиците на страниците съдържат 512 64-разрядни елементи. Регистърът CR3 съдържа старшите 27 разряда на PDPT. Старшите 2 разряда на линейния адрес (31-30) задават отместването на един от 4-те елемента в PDPT. Този елемент задава базов физически адрес на каталог на страници. Разряди 29-21 на линейния адрес избират елемент от каталога на страниците, който осигурява базовия адрес на таблица на страници, а разреди 20-12 индексират елемент в тази таблица.

В схемите за странично преобразуване за страниците с размер 4MB и 2MB отсъстват таблици на страници. Избраният от каталога PDE директно задава базовия адрес на страницата, а разреди 20-0 на линейния адрес определят отместването в нея.

Въпроси и задачи:

1. Стартирайте симулатора sim486. Заредете в него програма за изпълнение. В произволна точка от изпълнението превключете МП в защитен режим като заредите от прозореца на системните регистри подходящо значение в тях. Каква е реакцията на МП при изпълнение на следващата инструкция? На кои регистри трябва да бъде променено значението и как, за да се превключи МП в защитен режим?

2. Съдържанието на кои регистри трябва да бъде променено, за да продължи правилно изпълнението на заредената програма след превключване в защитен режим? Направете нужните промени в регистрите и евентуално в паметта от съответните прозорци на симулатора. Проверете правилността на решението.

3. Как се изчисляват адресите на операндите и инструкциите в защитен режим? Изчислете адресите на операндите в инструкциите на вашата програма и направете проверка с изчислените адреси от симулатора.

4. Каква ще бъде реакцията на процесора, ако в произволна точка от изпълнението на програмата разрешите механизма на странициране? Кои регистри трябва да бъдат променени и как, за да се разреши този механизъм? Опитайте да създадете необходимите структури в регистрите и паметта, за да продължи изпълнението на програмата с разрешено странициране.

5. Как при разрешен механизъм на странициране се определят физическите адреси на операндите и инструкциите в паметта? Обяснете влиянието на съдържанието на буферите на устройството за предварително извличане на правилността на изпълнение на програмата след разрешаване на страницирането. А на кеш паметта?